

Adeline. Madeline

Sieci liniowe

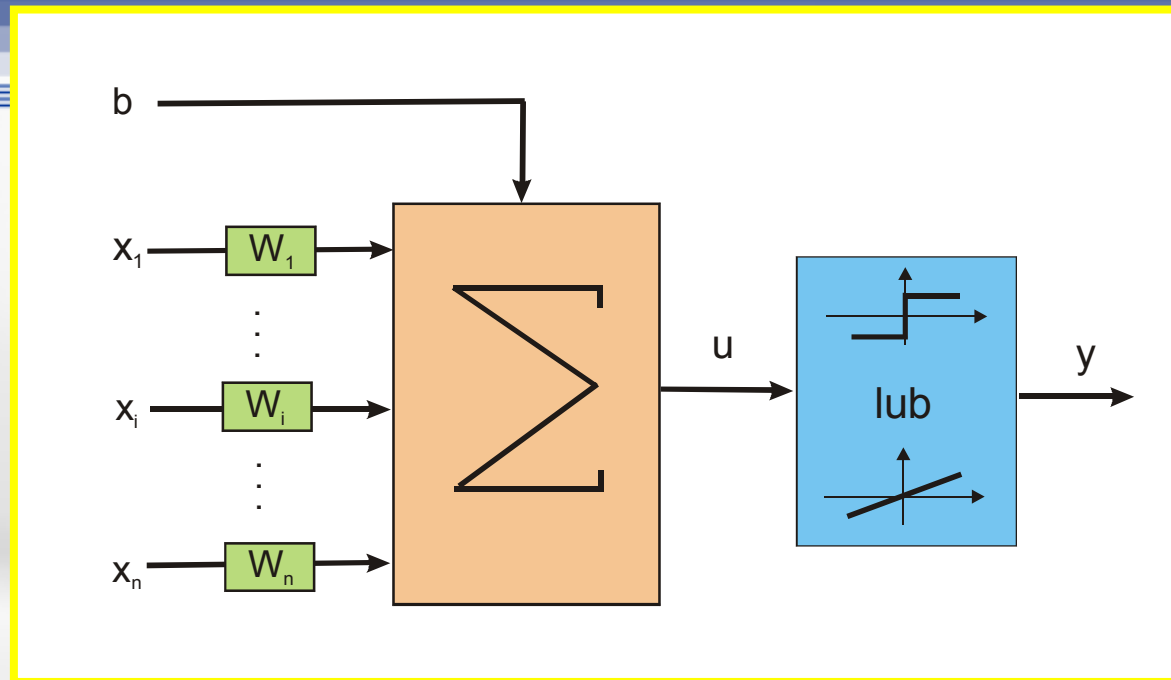
Neuron Adeline

- Literatura podaje dwie różne definicje struktury Adaline.
 - Pierwsza określa ją jako **element liniowy**, którego sygnał wyjściowy opisany jest następująco:

$$y = u = \sum_{i=0}^n w_i x_i$$

- indeks $i = 0$ dotyczy elementu progowego z doprowadzonym sygnałem $x_0=1$.
 - wprowadzenie współczynnika proporcjonalności k jest jedynie zmianą ilościową, a nie jakościową. W przypadku włączenia elementu Adaline w strukturę sieci wielowarstwowej jego rolę przejmują wagi połączeń wychodzących z elementu.
- Druga definicja określa strukturę Adaline identycznie jak **perceptron**, którego blok aktywacji realizujący bipolarną funkcję signum:

$$y = \begin{cases} 1 & \text{gdy } \sum_{i=0}^n w_i x_i > 0 \\ -1 & \text{gdy } \sum_{i=0}^n w_i x_i \leq 0 \end{cases}$$



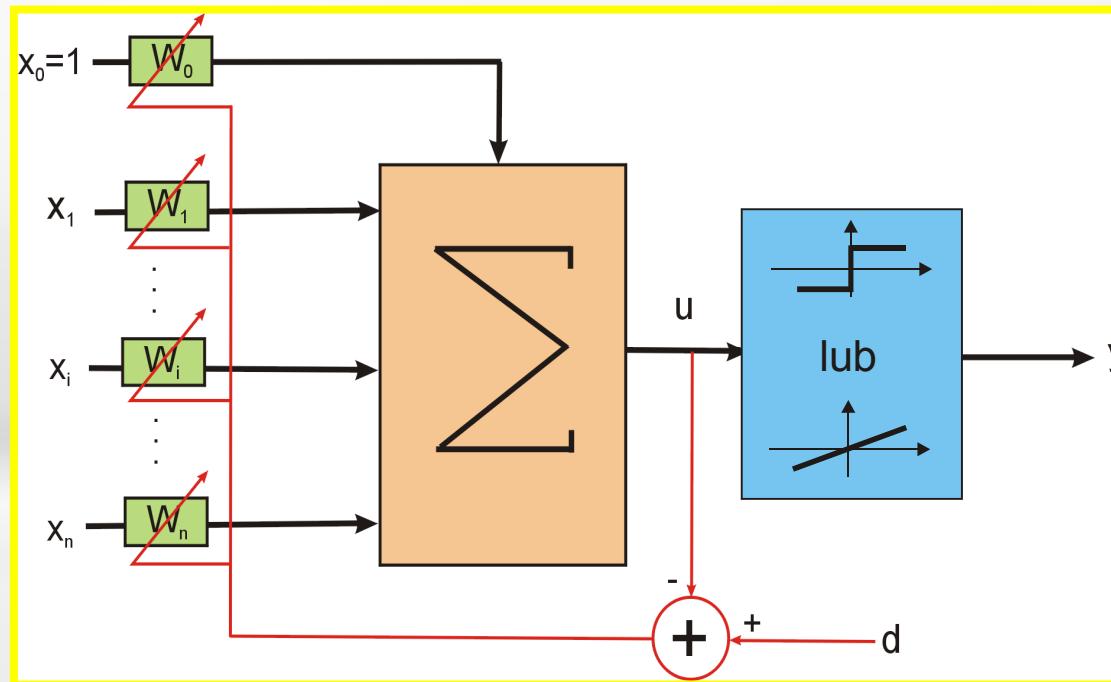
- Wyjście neuronu jest równe:
 - w przypadku liniowej funkcji przejścia

$$y = \sum_{i=0}^n w_i x_i = \mathbf{w}^t \mathbf{x}$$

- w przypadku bipolarnej funkcji przejścia

$$y = \operatorname{sgn}\left(\sum_{i=0}^n w_i x_i\right) = \operatorname{sgn}(\mathbf{w}^t \mathbf{x})$$

Uczenie neuronu Adeline

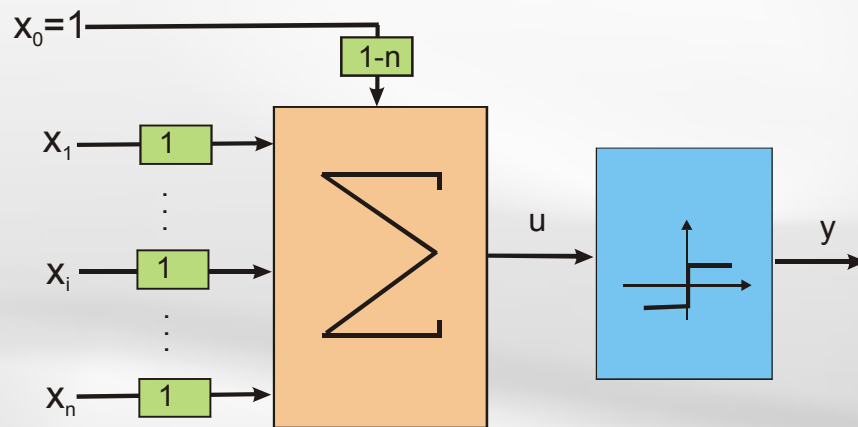


- Uczenie neuronów Adeline opiera się na minimalizacji błędu średniokwadratowego
- Wartością wpływającą na zmianę wag jest w przypadku:
 - perceptronu - wyjście neuronu
 - Adeline - potencjał membranowy

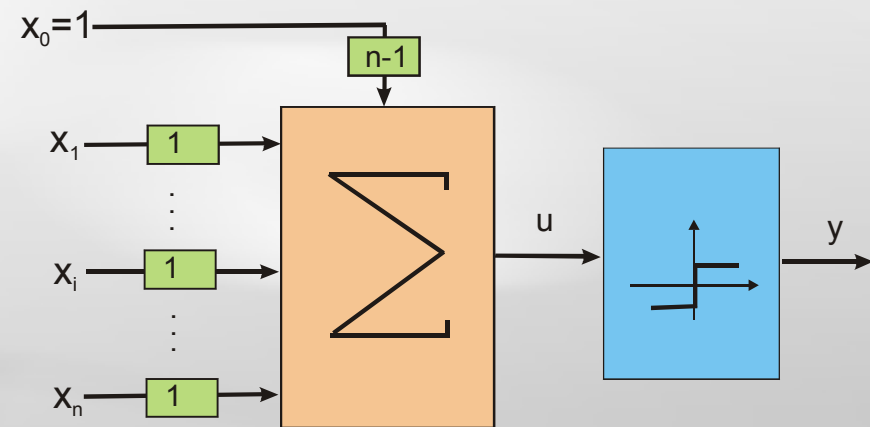
Madelin

- Adeline podlega podobnym ograniczeniom jak element perceptronowy
- W realizacjach praktycznych neuron **Adeline** najczęściej występuje w grupach, tworząc warstwę zwaną **Madeline** (ang. Many adaline).
- Każdy neuron warstwowy podlega uczeniu zgodnie z regułą Adaline.
- Wyjścia poszczególnych neuronów warstwy mogą być zorganizowane w różny sposób, charakterystyczne są trzy z nich
 - OR,
 - AND,
 - Majority,

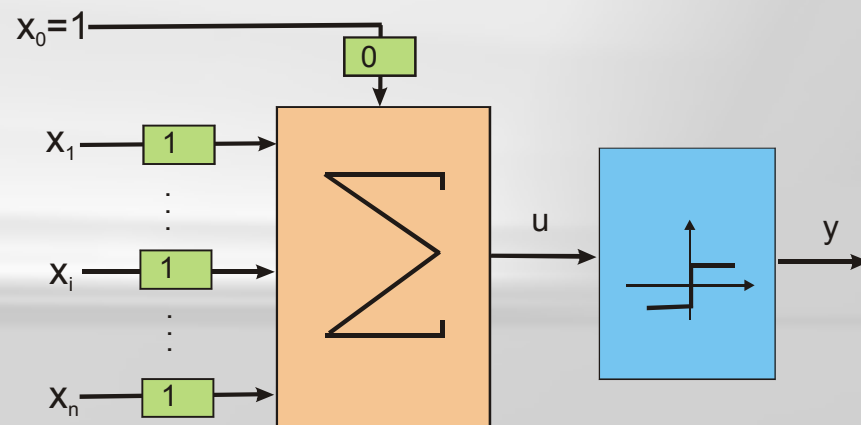
AND



OR

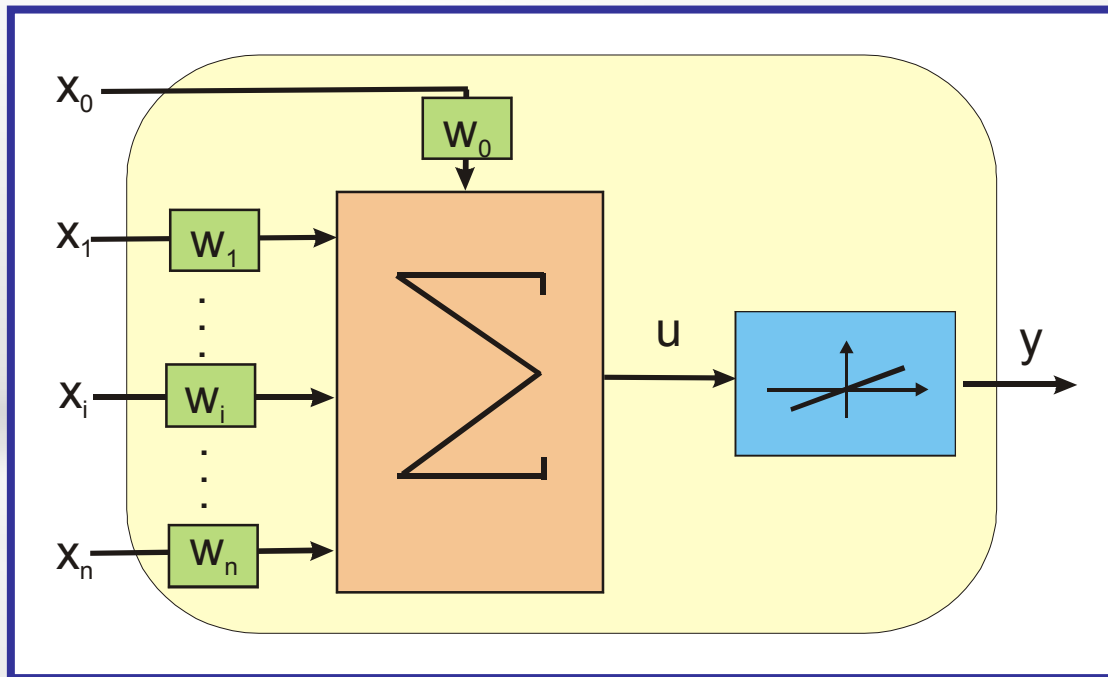


Majority



$$y = \begin{cases} 1 & \text{gdy } \sum_{i=0}^n w_i x_i > 0 \\ -1 & \text{gdy } \sum_{i=0}^n w_i x_i \leq 0 \end{cases}$$

Neuron liniowy



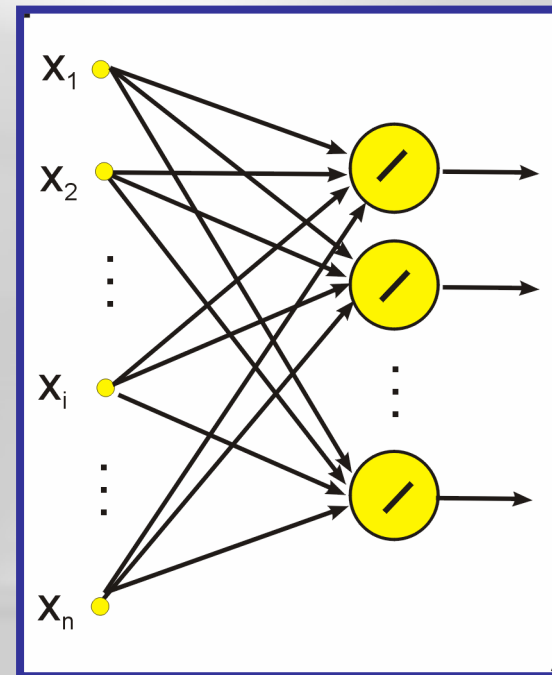
$$y = k \sum_{i=0}^n w_i x_i$$

- Neuron liniowy jest zdolny podzielić przestrzeń cech na dwie klasy (separowalne liniowo)
- Wiedzę zdobywa w procesie uczenia, korekta wag odbywa się zgodnie ze wzorem

$$w(k+1) = w(k) + \eta(d - y)x$$

Sieci liniowe

- Są to najprostsze ze wszystkich rodzajów sieci neuronowych.
- Sieci liniowe są zawsze sieciami jednowarstwowymi, bez warstwy ukrytej i dlatego pracują szybciej i szybciej się uczą.
- Sieć liniowa jednowarstwowa pracuje na zasadzie korelacji i ma znacznie mniejsze możliwości przekształceń danych wejściowych na dane wyjściowe.



- Zastosowania sieci liniowych
 - Filtracja
 - Proste zadania klasyfikacji, gdzie klasy są separowalne liniowo.
 - Zagadnienia predykcji (przewidywania) .
- Wady sieci liniowych
 - Nie ma sensu budowa wielowarstwowej sieci liniowej – w sensie funkcjonalnym daje się ona zawsze zastąpić siecią jednowarstwową
 - Rozwiązuje jedynie proste zadania klasyfikacji - (separowalność liniowa w połączeniu w jednowarstwowością funkcjonalną mocno ogranicza możliwości jej stosowania).

Zastosowania sieci liniowych

- Filtry tworzone za pomocą tych sieci mogą być wykorzystane do typowych zadań **filtracji sygnałów, eliminacji szumów, polepszeniu sygnału, wydobywaniu określonych cech sygnałów, analizy widmowej** itp.
 - Sieci liniowe są chętnie wykorzystywane jako filtry do eliminacji szumów w liniach telefonicznych.
- Filtry tego typu mogą służyć **do odtwarzania kompletnego sygnału na podstawie jego fragmentu**, np. odnalezienie w bazie danych obrazu na podstawie jego fragmentu. Takie zadania nazywa się pamięcią asocjacyjną w odróżnieniu od typowej pamięci komputera. Pamięć taka pozwala odtwarzać cały zestaw zapamiętanych informacji w przypadku dostarczenia niekompletnych danych.
- Innym zastosowaniem tych sieci jest tzw. filtr nowości, np. do **wyświetlania na ekranie tylko tych fragmentów sceny które uległy zmianie**. Może mieć to zastosowanie do ochrony obiektów przed włamaniem, oraz w automatyce komputerowej.

Matlab - tworzenie sieci liniowej

newlin Funkcja tworzy jednowarstwową sieć neuronową, złożoną z zadanej liczby neuronów o liniowych funkcjach aktywacji.

```
net = newlin(pr, s, id, lr)
```

wejścia

- `pr` - macierz (o rozmiarze $R \times 2$) zawierająca min i max dla R wejść sieci
- `s` - liczba neuronów w sieci (równa ilości wyjść z sieci);
- `id` - wejściowy wektor opóźnień, domyślnie = $[0]$;
- `lr` - współczynnik uczenia sieci; domyślnie = 0.001.

wyjście

`net` - obiekt opisujący architekturę utworzonej sieci neuronowej

```
net = newlin(pr, s, 0, P)
```

wejścia

- `pr` - macierz ($R \times 2$) zawierająca min i max wartości dla R wektorów wejściowych;
- `s` - liczba neuronów w sieci (równa ilości wyjść z sieci);
- `0` - wartość zero;
- `P` - macierz wektorów wejściowych (każda kolumna macierzy to jeden wektor wejściowy).

wyjście

`net` - obiekt opisujący architekturę utworzonej sieci neuronowej

newlind

Funkcja tworzy jednowarstwową sieć neuronową, złożoną z zadanej liczby neuronów o liniowych funkcjach aktywacji. Ponadto funkcja oblicza wagi **w** oraz wartości bias (progi) **b** neuronów sieci w wyniku rozwiązania metodą najmniejszych kwadratów równania liniowego:

$$[\mathbf{w} \ \mathbf{b}] * [\mathbf{P}; \text{ones}] = \mathbf{T}$$

```
net = newlind(P, T, pi)
```

wejścia

- P – macierz wektorów wejściowych (każda kolumna macierzy to jeden wektor wejściowy), macierz o wymiarach $R \times Q$, gdzie R jest liczbą wejść sieci (współrzędnych wektorów wejściowych); zaś Q – liczbą wektorów wejściowych, użytych do treningu sieci,
- T – macierz pożądaných odpowiedzi sieci (każda kolumna macierzy to jeden wektor oczekiwanych odpowiedzi), macierz o wymiarach $S \times Q$, gdzie S jest liczbą wyjść sieci; macierz zawiera Q pożądaných wektorów wyjściowych, odpowiadających wektorom zawartym w macierzy P.
- Pi – macierz o wymiarach $1 \times ID$, zawierająca początkowe wartości opóźnionych wejść sieci (ID jest liczbą opóźnień); jest to parametr opcjonalny, domyślnie $P_i = []$

wyjście

net – obiekt opisujący architekturę utworzonej sieci neuronowej

Zbudować i przeprowadzić uczenie neuronu z liniową funkcją aktywacji.
Dane wejściowe oraz oczekiwane odpowiedzi powinny być podawane przez użytkownika.

Uczenie w oparciu o regułę stosowana w neuronach Adeline:

```
P=input('Podaj macierz wejściową P=');  
T=input('Podaj macierz oczekiwanych odpowiedzi T=');  
net=newlin([-2 2],1); %utworzenie sieci liniowej  
net=train(net,P,T); %trening zbudowanej sieci  
a=sim(net,P); %zmienna a zawiera wynik nauczania neuronu  
blad=T-a  
W=net.IW{1,1} %W przechowuje wartości wag  
B=net.b{1} %B przechowuje wartości bias
```

Wagi wyliczane bezpośrednio:

```
P=input('Podaj macierz wejściową P = ');  
T=input('Podaj macierz oczekiwanych odpowiedzi T = ');  
net=newlind(P,T); %utworzenie sieci liniowej i wyliczenie wag  
a=sim(net,P); % symulacja sieci  
blad=T-a  
W=net.IW{1,1}  
B=net.b{1}
```

Matlab - symulacja pracy sieci

sim Funkcja wyznacza wartości wyjść sieci neuronowej dla zadanej macierzy danych wejściowych.

[y, Pf, Af, e, Perf] = sim(net, P, Pi, Ai, T)

wejścia

- net** - struktura (obiekt) zawierająca opis wielowarstwowej sieci jednokierunkowej
- P** - macierz wektorów wejściowych (każda kolumna to jeden wektor wejściowy),
- Pi** - macierz utworzona z wektorów początkowych opóźnień próbek sygnału wejściowego sieci; domyślnie = 0
- Ai** - macierz utworzona z wektorów początkowych opóźnień próbek sygnału w kolejnych warstwach sieci; domyślnie = 0
- T** - macierz pożądaných odpowiedzi sieci (każda kolumna to jeden wektor odpowiedzi)

wyjście

- y** - wartości wyjść wytrenowanej sieci dla wektorów zapisanych w **P**,
- Pf** - wyjściowa macierz utworzona z wektorów opóźnień próbek sygnału wejściowego sieci,
- Af** - wyjściowa macierz utworzona z wektorów opóźnień próbek sygnału w poszczególnych warstwach sieci,
- e** - wektory błędów popełnianych przez sieć podczas procesu uczenia,
- Perf** - wartość funkcji oceny błędu odwzorowania sieci, której nazwę zawiera pole `'net.performFcn'` (domyślnie `'mse'` – błąd średniokwadratowy)

Matlab - symulacja pracy sieci - przykłady

Przykład 1

```
% utworzenie perceptronu o 2 wejściach i 1 wyjściu
net = newp([0 1; 0 1],1);
% Symulacja pracy perceptronu dla trzech różnych wektorów
% uczących (pokazano różne sposoby podawania danych)
p1 = [.2; .9]; a1 = sim(net, p1)
p2 = [.2 .5 .1; .9 .3 .7]; a2 = sim(net, p2)
p3 = { [.2; .9] [.5; .3] [.1; .7] }; a3 = sim(net, p3)
```


Przykład 2

```
% utworzenie sieci liniowej
net = newlin([0 2; 0 2; 0 2], 2, [0 1]);
p1 = {[2; 0.5; 1] [1; 1.2; 0.1]};
% obliczenie wyjść sieci dla dwóch zadanych wektorów
% wejściowych (przy założeniu zerowych opóźnień na
% wejściach):
[y1, pf] = sim(net, p1) p2 = {[0.5; 0.6; 1.8]...
    [1.3; 1.6; 1.1] [0.2; 0.1; 0]};
% Wywołanie wykorzystuje wartości opóźnień, wyznaczone w
% poprzednim wywołaniu funkcji:
[y2, pf] = sim(net, p2, pf)
```